

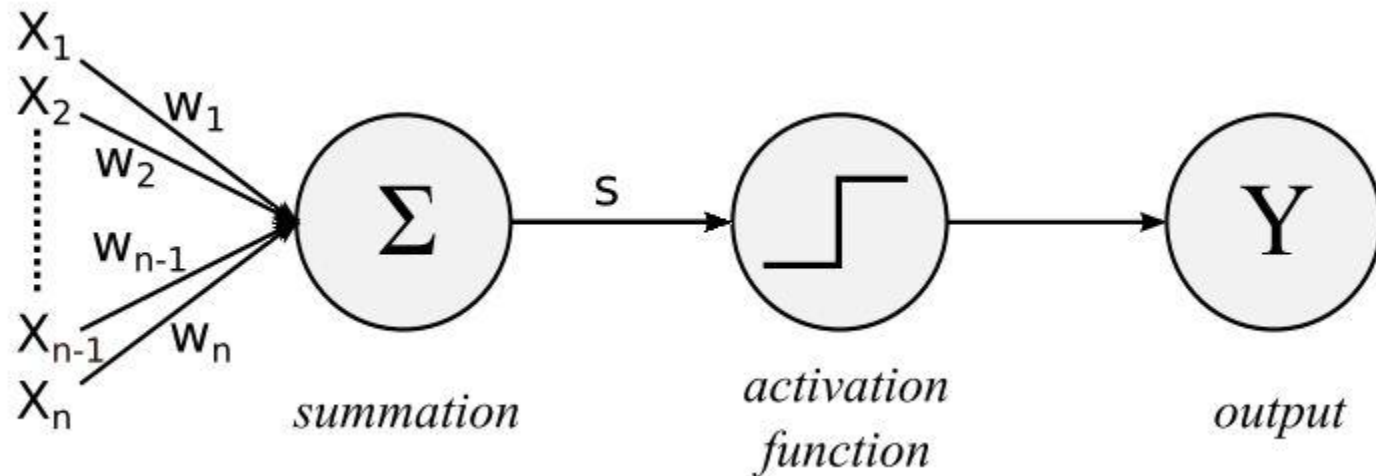
Parallel Computing For Neural Networks

Dan Grau and Nick Sereni

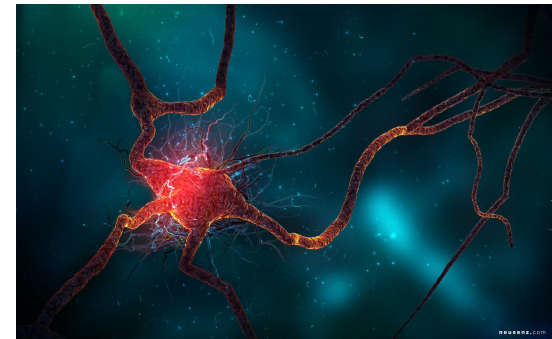
Introduction

- What is a neural network?
 - Collection of interconnected *neurons* that compute and generate impulses
 - Components of a neural network include *neurons*, *synapses*, and *activation functions*
 - Neural networks modeled by mathematical or computer models are referred to as *artificial* neural networks
 - Neural networks achieve functionality through *learning/training functions*

Neurons, Synapses, & Activation

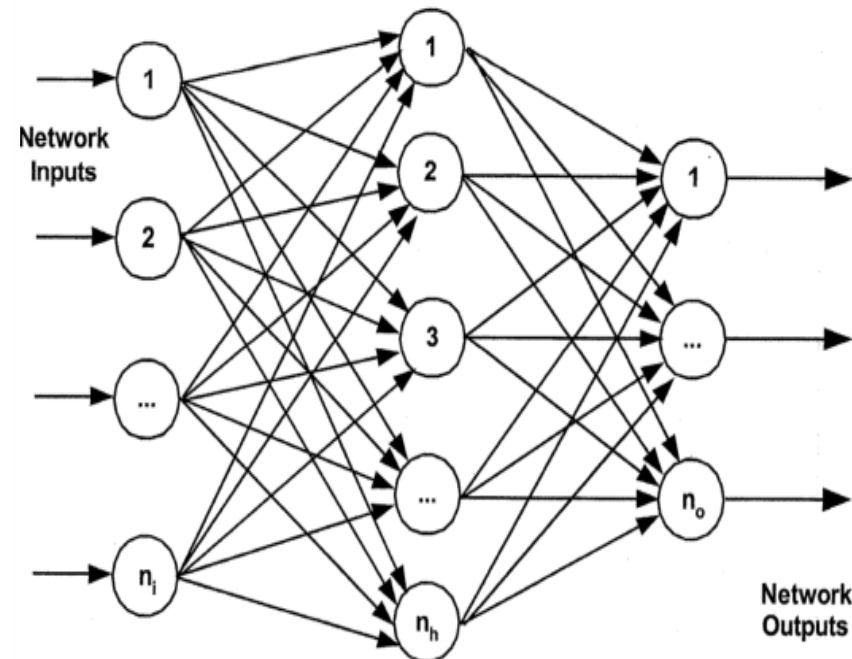
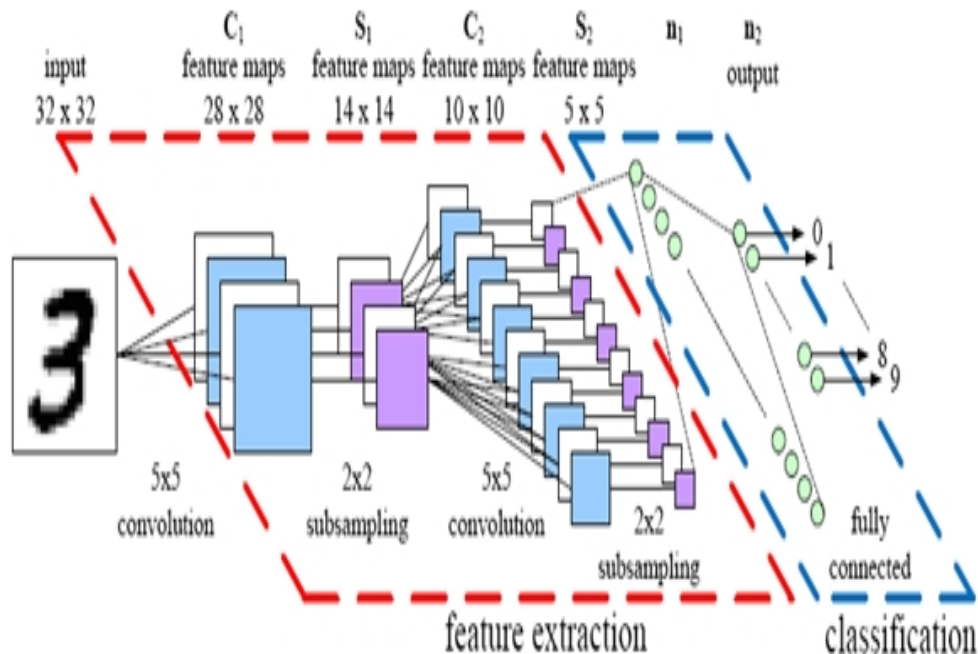


A neuron is commonly modeled by synapses, represented by W_i , connected to input signals, represented by X_i . These inputs are either inputs to the network, or outputs from previous neurons. A neuron will sum these inputs and calculate the output based on an activation function.



Network Topologies

- Network topologies describe how neurons map to each other. Many artificial neural networks (ANNs) utilize network topologies that are very parallelizable.



Most Commonly Used Algorithms

- **Multilayer feedforward networks**
 - Layered network where each layer of neurons only outputs to next layer
- **Feedback networks**
 - Single layer of neurons with outputs that loops back to the inputs
- **Self-organizing maps (SOM)**
 - Forms mappings from a high dimensional space to a lower dimensional space (one layer of neurons)
- **Sparse distributed memory (SDM)**
 - Special form of a two layer feedforward network that works as an associative memory

Learning/Training

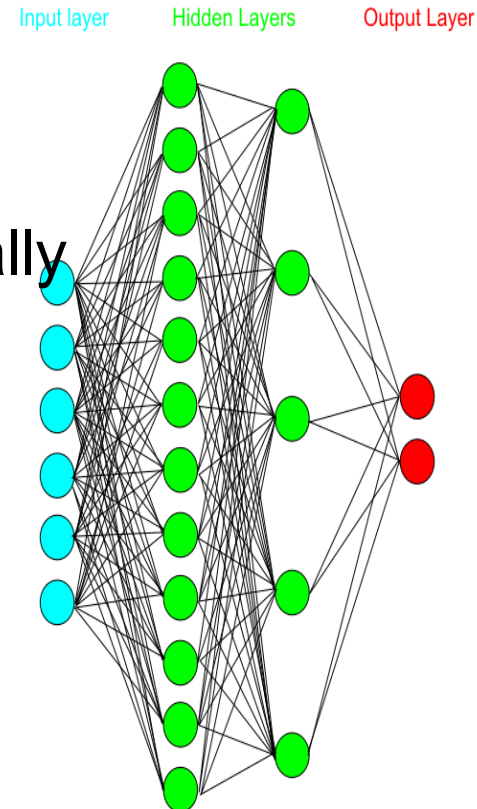
- The key to neural networks is weights. Weights determine the importance of signals and essentially determine the network output.
- Training cycles adjust weights to improve the performance of a network. The performance of an ANN is *critically dependant* on training performance.
- An untrained network is basically useless.
- Different training algorithms lend themselves better to certain network topologies, but some also lend themselves to parallelization

Why Neural Networks?

There are about 10^{16} synapses in the human brain each performing multiple operations/s. This means that the brain is processing approximately 10^{16} operations/s while only dissipating a few watts. ANNs aim to mimic this behavior in hopes of achieving similar performance. The brain's ability to interpret raw sensory data and perform behavior based on it outperform any modern design.

Why Parallel Computing?

- Training and evaluation of each node in large networks can take an incredibly long time
- However, neural networks are "embarrassingly parallel"
 - Computations for each node are generally independent of all other nodes



Possible Levels of Parallelization

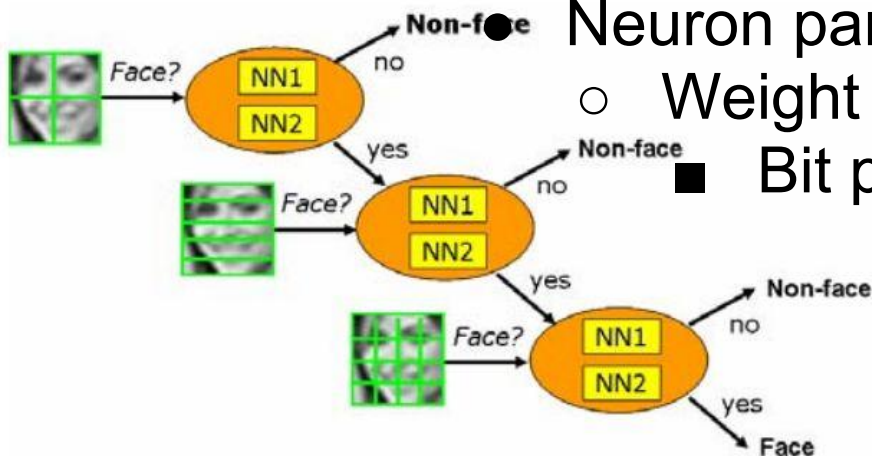
Typical structure of an ANN:

- For each training session
 - For each training example in the session
 - For each layer (forward and backward)
 - For each neuron in the layer
 - For all weights of the neuron
 - For all bits of the weight value

Possible Levels of Parallelization

So there are at least 6 possible ways to achieve parallelism, from coarse grain to fine grain:

- Training session parallelism
 - Training example parallelism
 - Layer parallelism or Forward-Backward parallelism
 - Neuron parallelism
 - Weight parallelism
 - Bit parallelism

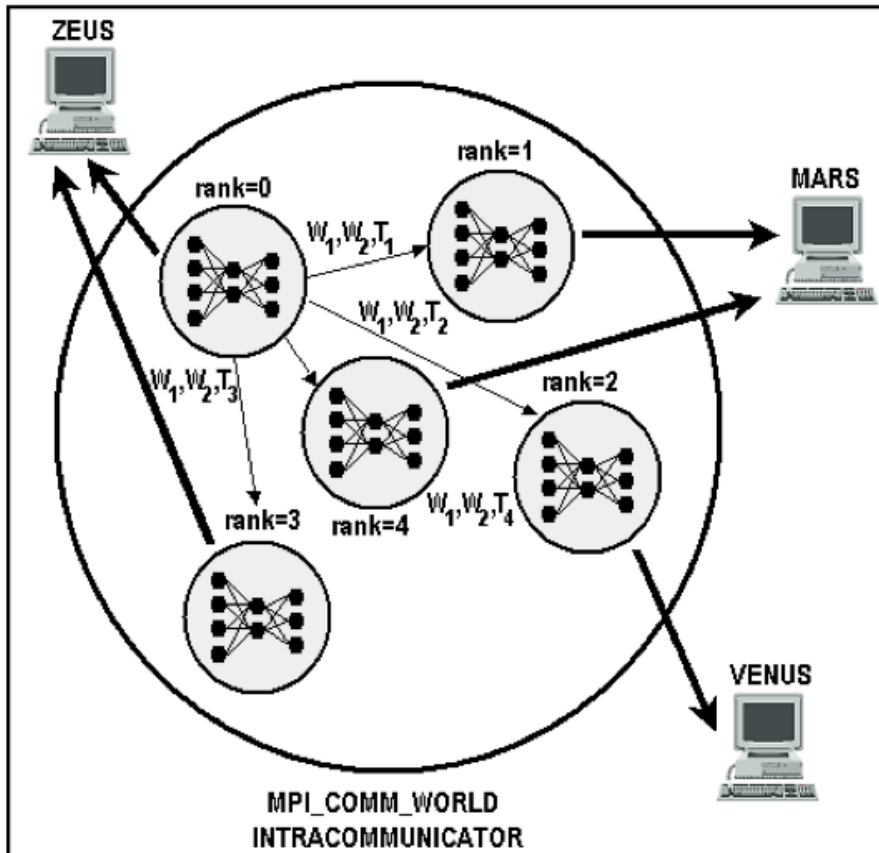


Even coarser parallelism: Network level [4]

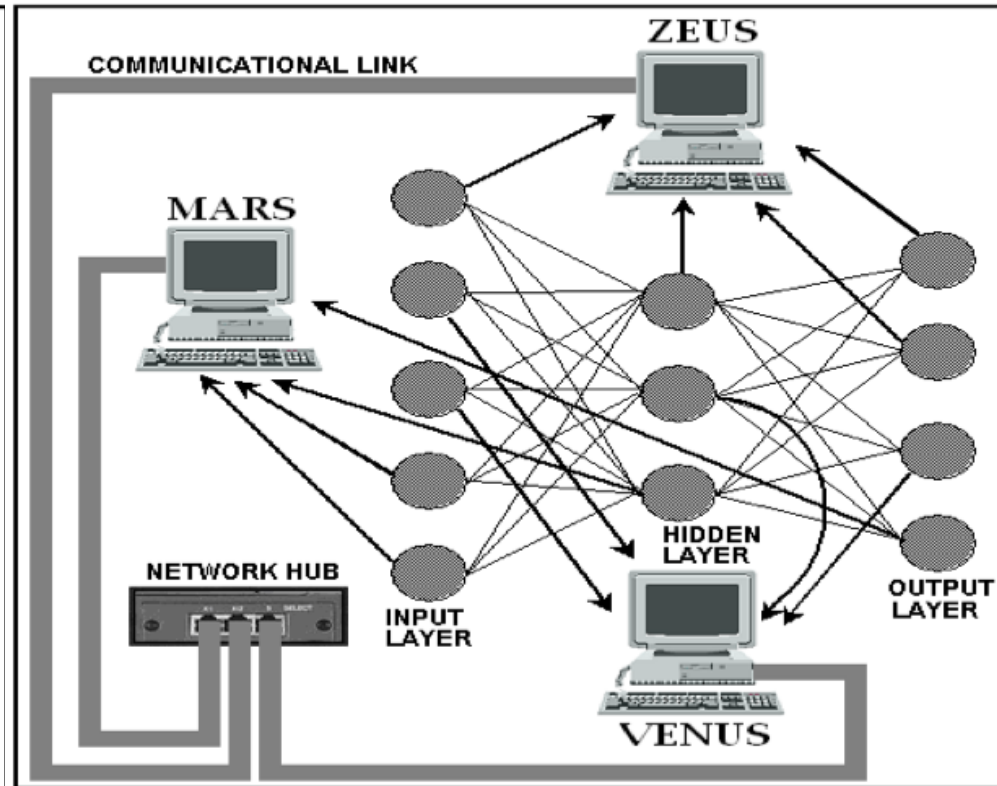
Matching the Degree of Parallelism to the Network

- The multiple layer structure of many feedforward networks and their back-propagation training technique severely limits the speedup of parallel neuron computations
 - Training set or network level parallelism more practical
- Neuron level parallelism more effective for single layer networks
 - SOM or some recurrent networks

Network Level vs Neuron Level Parallelism



Network Level Parallelism [7]



Neuron Level Parallelism [7]

Computational Considerations

- Main operation is matrix-vector multiplication
 - For an N-node layer, there are N^2 scalar multiplications and N sums of N numbers
 - Need good support for multiply or multiply-and-add
- Sigmoid function
 - In many models, neuron output is determined by a function such as $f(x) = 1/(1+e^{-x})$
 - For efficiency, needs to be approximated via lookup table or a piecewise linear model

Communication Considerations

- High degree of connectivity and large data flows are characteristic features of ANNs
- Structure and bandwidth of communication are very important
 - Depends of degree of parallelism
- Broadcast or ring communication very efficient for ANNs

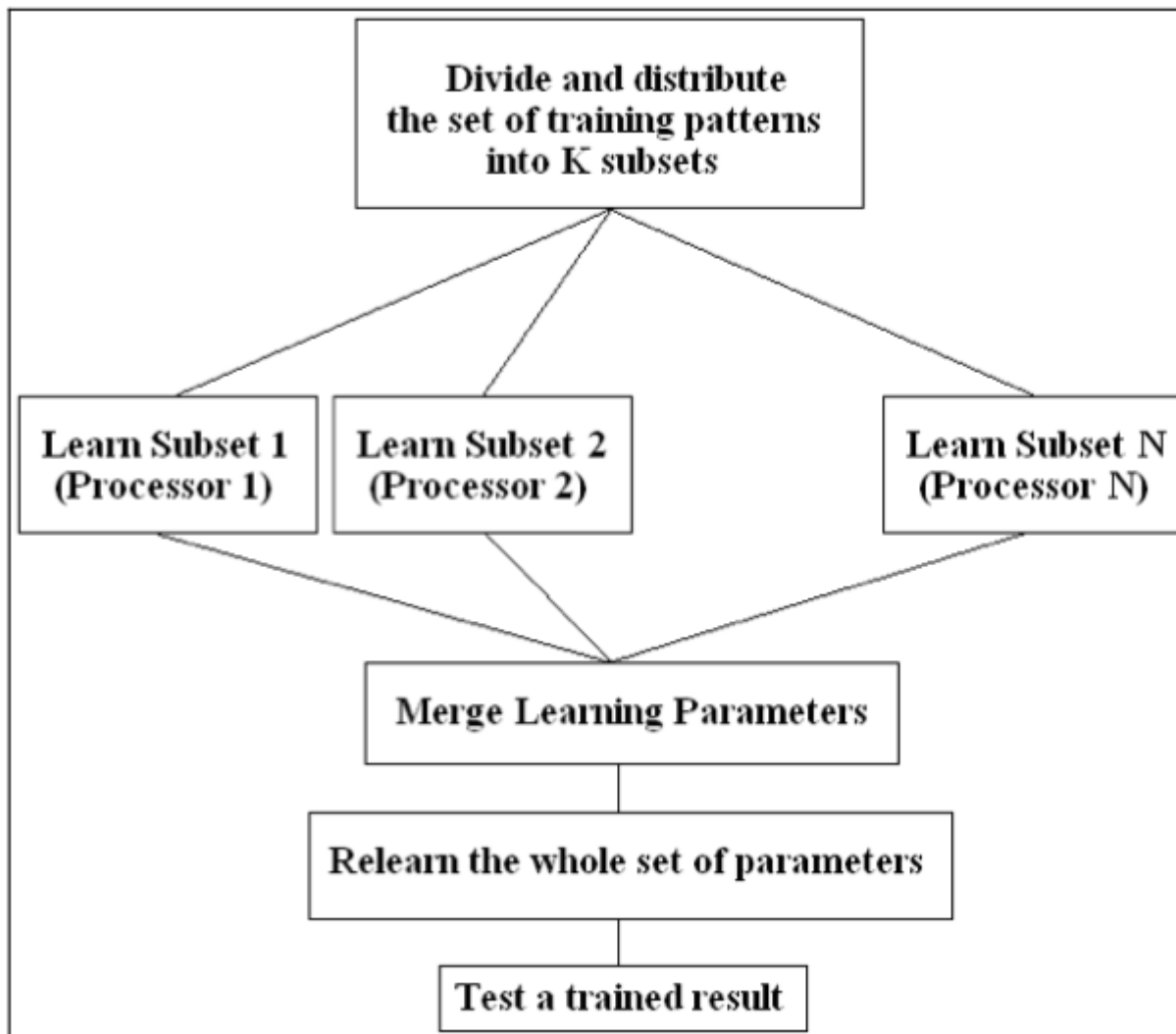
More processors is not always better

Used Processors	1	2	3	4	5	6	7	8	9	10	11	12	13
neuron distribution among available processors	1x 25	1x 13 1x12	1x 9 2x8	1x 7 3x6	5x 5	1x 5 5x4	4x 4 3x3	1x 4 7x3	7x 3 2x2	5x 3 5x2	3x 3 8x2	1x 3 11x2	12x 2 1x1
Used Processors	14	15	16	17	18	19	20	21	22	23	24	25	
neuron distribution among available processors	11x 2 3x1	10x 2 5x1	9x 2 7x1	8x 2 9x1	7x 2 11x1	6x 2 13x1	5x 2 15x1	4x 2 17x1	3x 2 19x1	2x 2 21x1	1x 2 23x1	25x 1	

Division of labor for a 5x5 SOM (25 neurons). Bold numbers show the required number of sequential loops [5]

Additionally, the effort and time to transfer data between processors increases with each step

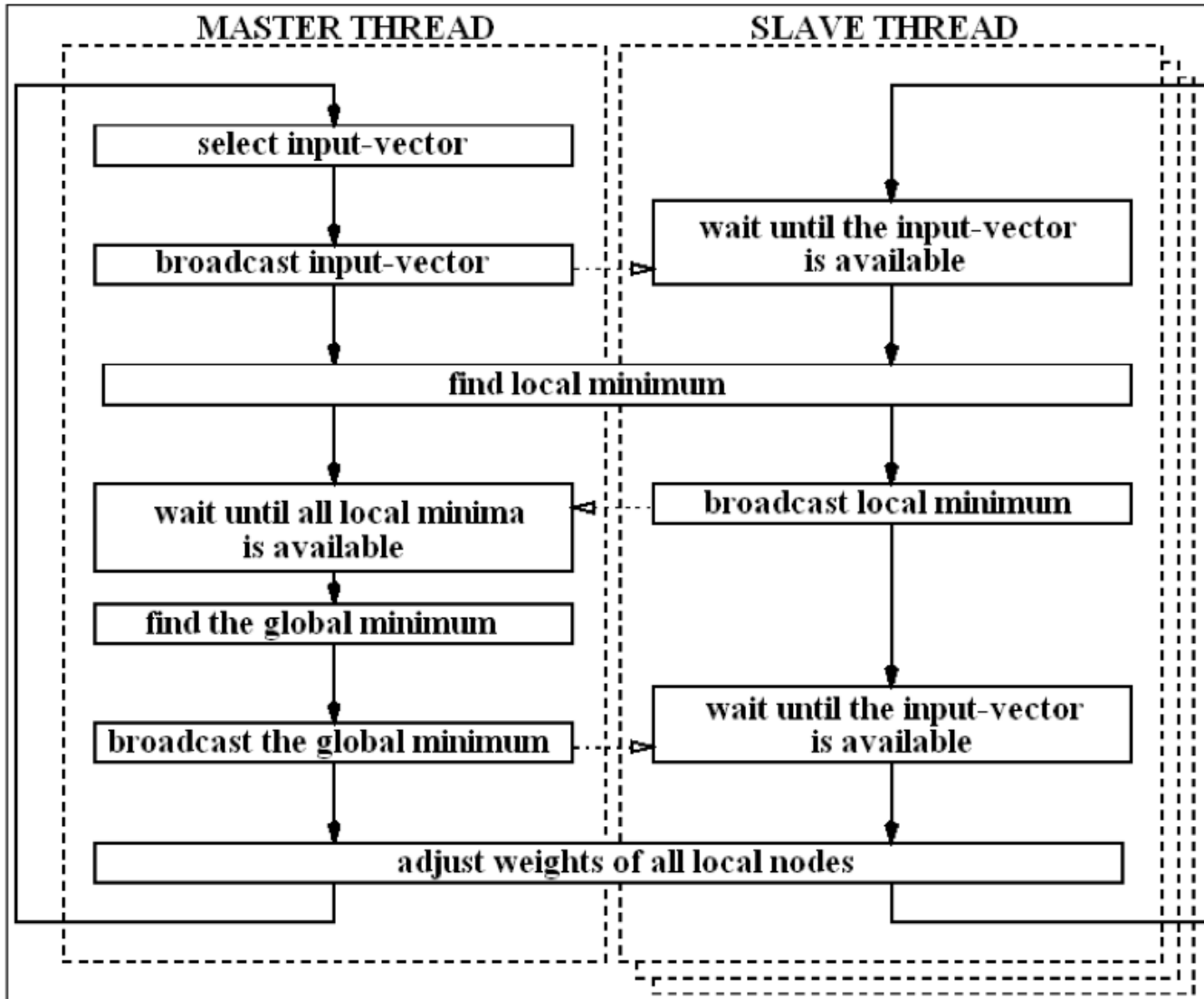
Example: Parallel Back-propagation training [7]



Training-set parallelization

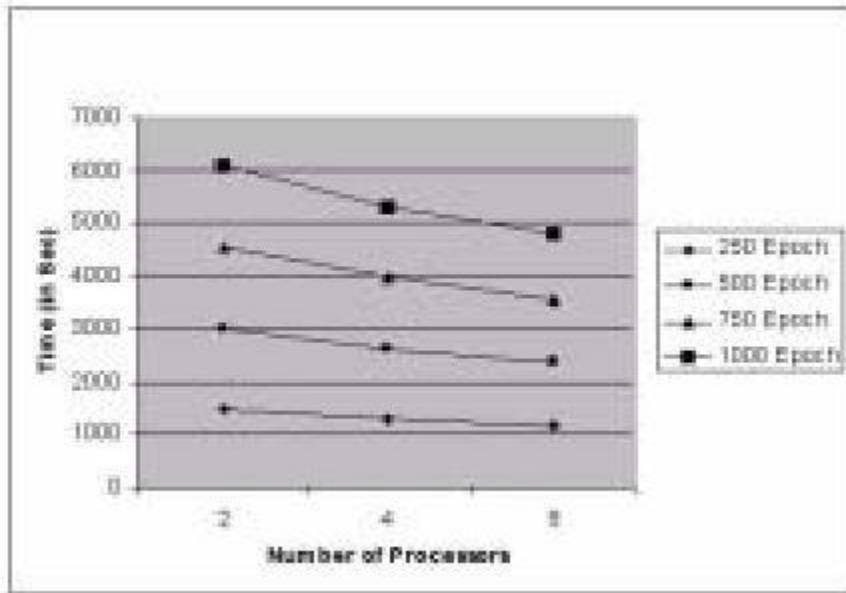
- Each processor performs full sequential training of the network for one portion of the training set
- Main processor performs averaging procedure on results to extract final weight table

Example: Parallel SOM [7]

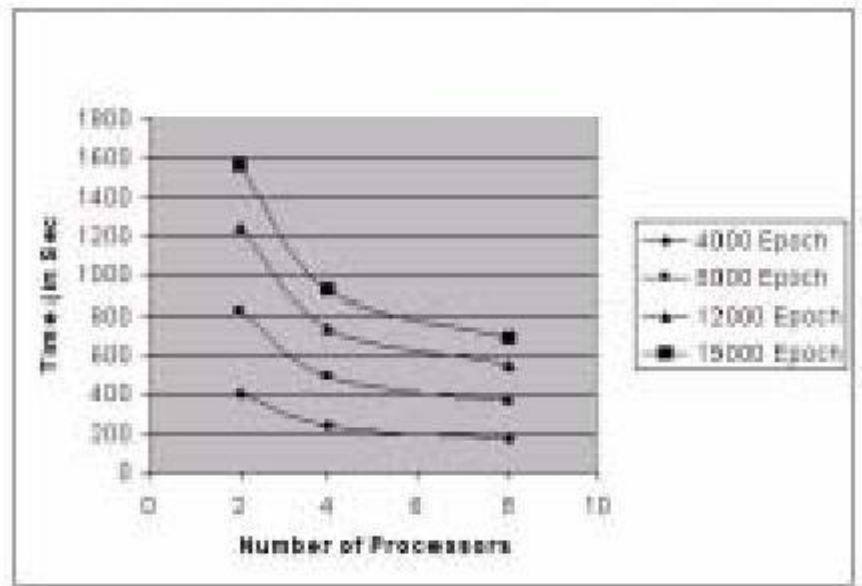


Partitions the neuron grid into a set of nodes that can be processed independently of each other.

Results

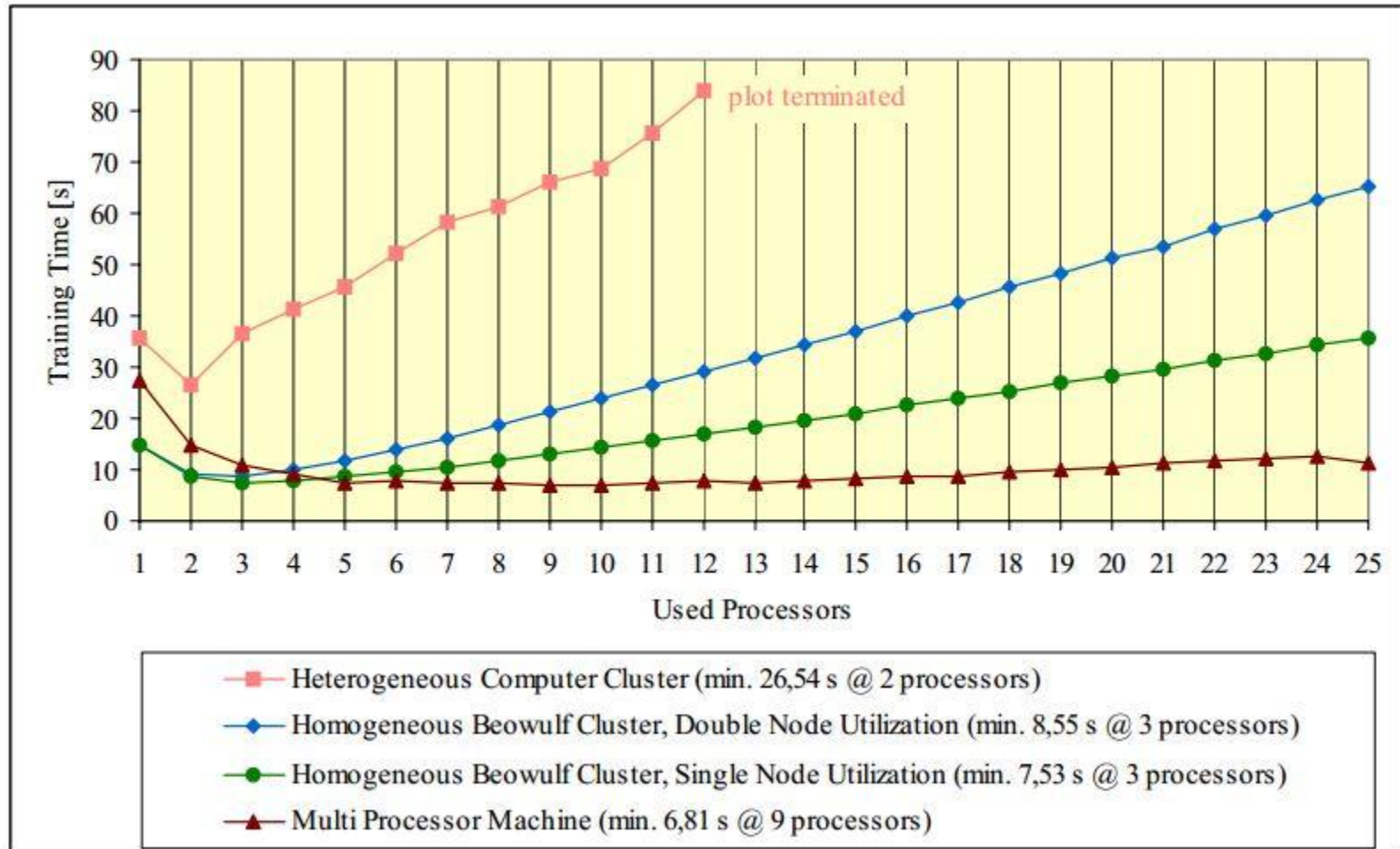


Execution time vs. number of processors in MPI:
Neuron-level parallelism[3]



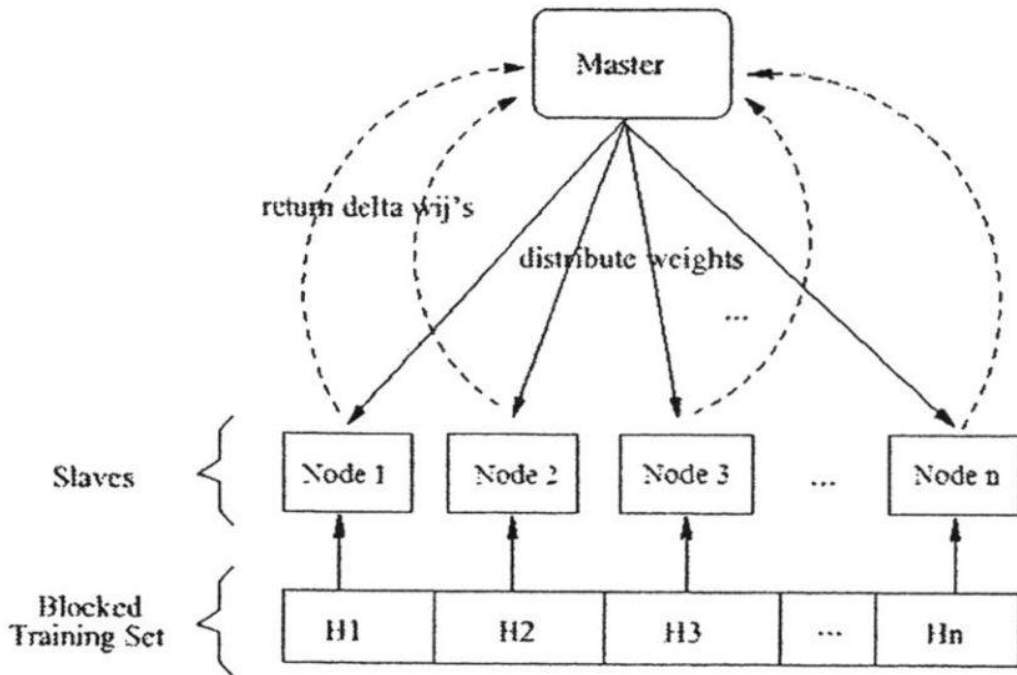
Execution time vs. number of processors in MPI:
Training-Set Parallelism[3]

Results (cont...)

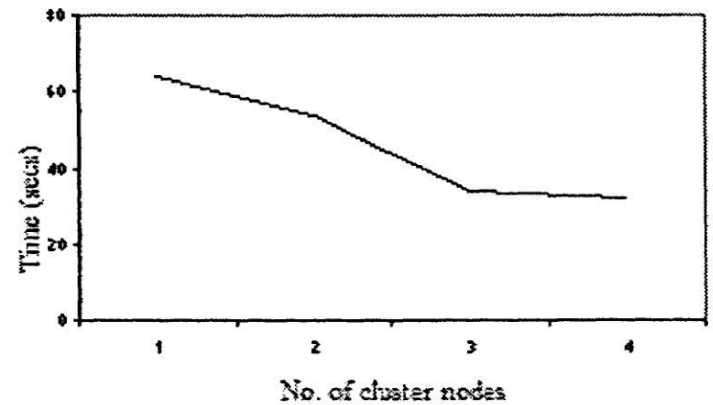
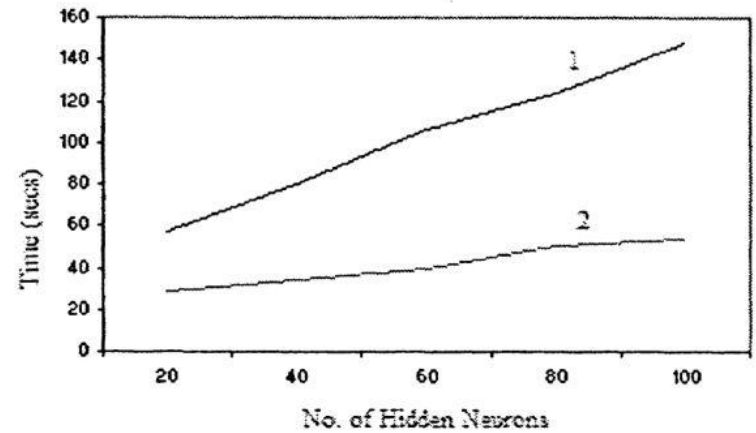


Training Time of 25-node SOM depending on the number of used processors [5]

Results (cont...)



Problem Decomposition [6]



References

- [1] Kwabena Boahen. Neuromorphic microchips. *Scientific American*, 292(5):56–63, 2005.
- [2] Nordström, T. and B. Svensson, “Using and designing massively parallel computers for artificial neural networks,” *Journal of Parallel and Distributed Computing*, vol. 14, no. 3, pp. 260-285, 1992.
- [3] Thulasiram, R.K.; Rahman, R.M.; Thulasiraman, P., "Neural network training algorithms on parallel architectures for finance applications," *Parallel Processing Workshops, 2003. Proceedings. 2003 International Conference on* , vol., no., pp.236,243, 6-9 Oct. 2003
- [4] Turchenko, V.; Paliy, I.; Demchuk, V.; Smal, R.; Legostaev, L., "Coarse-Grain Parallelization of Neural Network-Based Face Detection Method," *Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications, 2007. IDAACS 2007. 4th IEEE Workshop on* , vol., no., pp. 155,158, 6-8 Sept. 2007
- [5] Seiffert, Udo. "Artificial neural networks on massively parallel computer hardware." *Neurocomputing* 57 (2004): 135-150.
- [6] Srinivasan, N.; Vaidehi, V., "Cluster computing for neural network based anomaly detection," *Networks, 2005. Jointly held with the 2005 IEEE 7th Malaysia International Conference on Communication., 2005 13th IEEE International Conference on* , vol.1, no., pp.5 pp., 0-0 0
- [7] A. Margaris, E. Kotsialos, and M. Roulmeliotis, “Simulating Parallel Neural Networks In Distributed Computing Systems,” *2nd International Conference “From Scientific Computing to Computational Engineering”*, pp. 5–8, 2006.